

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Курсовое проектирование по управлению ресурсами суперэвм

Отчет по курсовой работе

«Поиск числа треугольников и числа окружностей на
полигоне»

Студент
группы 5130201/30101

_____ Шишмаков В.О.

Преподаватель

_____ Курочкин М. А.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

1 Введение	3
2 Постановка задачи	4
3 Аппаратно программная платформа Nvidia CUDA	5
3.1 Архитектура Nvidia CUDA	5
3.1.1 Архитектура Blackwell (март 2024)	6
3.1.2 Архитектура Hopper (март 2022)	7
3.1.3 Архитектура Ады Лавлейс (сентябрь 2022)	8
3.1.4 Архитектура Ampere (2020)	10
3.1.5 Архитектура Pascal (2016)	12
3.1.6 Архитектура Maxwell (2014)	13
3.1.7 Архитектура Tesla (2006)	15
3.2 Вычислительные возможности Nvidia CUDA	17
3.3 Поточковая модель	19
3.4 Устройство памяти	20
3.5 Модели памяти	20
3.6 Модель вычислений на GPU	22
3.7 Компиляция программы	22
3.8 Планировщик задач	23
4 Суперкомпьютерный центр «Политехнический»	24
4.1 Состав	24
4.2 Характеристики	24
4.3 Технология подключения	25
5 Постановка решаемой задачи	27
6 Алгоритм решения задачи	28
6.1 Глобальная память	29
6.2 Разделяемая память	29
7 Описание эксперимента	31
8 Анализ результатов	32
Заключение	42

1 Введение

В настоящее время множество научных и инженерных задач требуют значительных вычислительных ресурсов, поскольку для их решения необходимо за приемлемое время выполнить огромное количество математических операций. Одним из способов справиться со сложными вычислениями служит параллельное программирование.

Суперкомпьютеры — это многопроцессорные вычислительные системы, способные производить триллионы операций в секунду. Благодаря им становятся возможными задачи, которые недоступны при традиционных последовательных вычислениях. Это особенно важно для таких областей, как численное моделирование, обработка изображений и машинное обучение. В связи с этим в последние годы широкое распространение получили вычисления на графических ускорителях — устройствах с массово-параллельной архитектурой. Для вычислений общего назначения можно использовать видеокарты с архитектурой Nvidia CUDA.

Примером мощной вычислительной системы служит суперкомпьютерный центр «Политехнический». Часть его узлов оснащена графическими ускорителями Nvidia Tesla K40X.

2 Постановка задачи

Цель работы:

Изучение методов параллельного программирования на базе архитектуры NVIDIA CUDA применительно к задаче вычисления числа треугольников и числа окружностей на полигоне $P(n \times n)$ с задействованием вычислительных мощностей суперкомпьютерного центра «Политехнический».

Задачи:

1. Освоить программно-аппаратную платформу NVIDIA CUDA: её архитектурные особенности, вычислительные возможности, организацию памяти и модель параллельных вычислений.
2. Изучить состав и технические характеристики вычислительных узлов СКЦ «Политехнический», а также освоить и применить на практике методику подключения к данному центру.
3. Разработать на языке CUDA C алгоритм поиска числа объектов на полигоне $P(n \times n)$.
4. Провести серию вычислительных экспериментов на узле «Торнадо» СКЦ «Политехнический», варьируя следующие параметры:
 - радиус окружности;
 - координаты вершин треугольника в локальной системе координат;
 - размер полигона.
5. Проанализировать полученные в ходе экспериментов данные и сопоставить время выполнения реализации при разных наборах параметров.

3 Аппаратно программная платформа Nvidia CUDA

NVIDIA CUDA (Compute Unified Device Architecture) представляет собой совокупность программных и аппаратных средств, разработанную компанией NVIDIA и предназначенную для организации параллельных вычислений на графических процессорах (GPU), а также для ускорения обработки информации.

3.1 Архитектура Nvidia CUDA

Архитектура графического процессора (GPU) включает:

- **GPU-процессор** — представляет собой массив потоковых процессоров (Streaming Processor Array).
- **Потоковый процессор** — состоит из кластеров текстурных процессоров (Texture Processor Clusters).
- **Текстурные процессоры** — в свою очередь, состоят из набора мультипроцессоров (Streaming Multiprocessors, SM).
- **Мультипроцессоры** — содержат несколько потоковых процессоров (Streaming Processors), которые также называют ядрами CUDA (CUDA cores).

Управление выполнением команд осуществляется с помощью GigaThread Engine, который распределяет блоки потоков между мультипроцессорами. Общая структура GPU не претерпевает кардинальных изменений при переходе от одной микроархитектуры к другой — в основном варьируются объём и скорость L2-кэша.

В архитектуре NVIDIA CUDA используется модель исполнения SIMT (Single Instruction Multiple Thread — одиночная инструкция, множество потоков). Эта модель сочетает черты MIMD (Multiple Instruction Multiple Data) и SIMD (Single Instruction Multiple Data). Вычислительная задача разбивается на множество блоков. Планировщик назначает блоки на мультипроцессоры (SM), причём каждый блок выполняется только на одном SM, однако на одном SM может одновременно выполняться несколько блоков.

Каждый блок состоит из потоков (threads). При выполнении потоки группируются в варпы (warps) — коллекции по 32 потока. Все потоки внутри одного варпа одновременно исполняют одну и ту же команду.

3.1.1 Архитектура Blackwell (март 2024)

Архитектура Blackwell представляет собой значительный шаг вперед в развитии графических технологий от NVIDIA, охватывая три ключевых аспекта применения: гейминг, студийные задачи и искусственный интеллект. Рассмотрим подробно каждый аспект, подчеркивая уникальные технологии и преимущества архитектуры.

Видеокарты серии Blackwell уже доступны в продаже. Надёжным партнёром по их поставке выступает интернет-магазин Telemart, предлагающий широкий ассортимент моделей NVIDIA GeForce RTX 50-й серии, включая как игровые, так и профессиональные решения.

Архитектура Blackwell делает особый акцент на реалистичную визуализацию человеческой кожи в играх. Технология RTX Skin использует усовершенствованные алгоритмы подповерхностного рассеивания света (SSS), что позволяет добиться правдоподобной передачи полутонов, текстур и цветовых переходов. В отличие от предыдущих реализаций, система теперь учитывает не только основной свет, но и контекстные изменения освещения — в том числе влияние окружающей среды и отражённого света от других поверхностей.

Особое внимание уделено адаптивным алгоритмам, которые в реальном времени настраивают интенсивность и глубину рассеивания света под кожей. Это позволяет сохранить корректное отображение даже при резкой смене условий — например, при переходе из солнечного пространства в тень или при использовании источников холодного направленного света. Эффект особенно заметен в сценах с близкими планами, где важны микродетали текстуры кожи — поры, сосуды, тональные переходы.

Первое практическое применение RTX Skin можно увидеть в Half-Life 2 RTX Remaster, где технология используется для обновлённого отображения лиц главных персонажей. В катсценах и диалогах заметно, как кожа реагирует на освещение от фонарей, солнечного света или светящихся экранов. Такой уровень фотореализма делает старую игру визуально современнее и подчёркивает потенциал RTX Skin в кинематографичных проектах нового поколения.

DLSS 4 с Frame Generation и Multi-Frame Generation

DLSS 4 представляет собой масштабное обновление технологии Deep Learning Super Sampling от NVIDIA, анонсированное одновременно с архитектурой Blackwell и видеокартами серии GeForce RTX 50. Основными нововведениями стали использование модели трансформера и интеграция Multi Frame Generation, благодаря которым достигается значительный прирост производительности и резкое улучшение визуального каче-

ства — особенно в динамичных сценах и при высоких разрешениях.

Новая система генерации кадров теперь способна учитывать не только текущий и предыдущий кадры, но и пространственно-временные зависимости между несколькими отрезками рендеринга, что минимизирует артефакты и повышает чёткость изображения. DLSS 4 также задействует более сложную нейросетевую архитектуру, позволяющую адаптироваться к конкретной игре и сценарию, обеспечивая более стабильный результат.

3.1.2 Архитектура Norper (март 2022)

Архитектура Norper представляет собой высокопараллельную вычислительную платформу, спроектированную на базе графического процессора GH100. Структура архитектуры опирается на сложную иерархическую организацию вычислительных блоков и глубоко интегрированную подсистему памяти, ориентированную на максимизацию локальности данных.

Вычислительное ядро и потоковые мультипроцессоры (SM)

Базовым строительным блоком архитектуры выступает потоковый мультипроцессор (Streaming Multiprocessor, SM). Каждый SM содержит независимые конвейеры для целочисленных вычислений (INT32) и вычислений с плавающей запятой, а также специализированные блоки матричной математики.

- **Тензорные ядра (Tensor Cores):** Вычислительные блоки, аппаратно реализующие операции умножения и сложения матриц (Matrix Multiply-Accumulate). Они аппаратно поддерживают вычисления смешанной точности, включая форматы FP8, FP16, bfloat16, TF32 и FP64.
- **Transformer Engine:** Аппаратный модуль, функционирующий совместно с тензорными ядрами на уровне SM. Он выполняет эвристический анализ распределения значений в тензорах на каждом слое нейронной сети и осуществляет динамическое (на лету) переключение точности вычислений между форматами FP8 и FP16, оптимизируя информационную энтропию и пропускную способность.
- **Инструкции DPX:** Интегрированный в арифметико-логические устройства (ALU) набор инструкций, реализующий аппаратное ускорение рекурсивных алгоритмов динамического программирования на уровне отдельных тактов процессора.

Иерархия памяти и асинхронная передача данных

Для предотвращения простоя вычислительных блоков (data starvation), архитектура Norper включает специализированные механизмы управления памятью:

- **Тензорный ускоритель памяти (Tensor Memory Accelerator, ТМА):** Выделенный аппаратный блок управления прямым доступом к памяти. ТМА берет на себя задачи асинхронного копирования многомерных блоков данных между глобальной памятью (HBM3) и разделяемой памятью (Shared Memory). Он вычисляет адреса и управляет транзакциями аппаратно, полностью высвобождая регистры и ALU потокового мультипроцессора.
- **Распределенная разделяемая память (Distributed Shared Memory, DSMEM):** Архитектурная инновация, позволяющая объединять пулы локальной разделяемой памяти нескольких SM. Мультипроцессоры могут напрямую читать и писать данные в L1/Shared память соседних SM по выделенному высокоскоростному интерконнекту, минуя обращение к кэшу второго уровня (L2) или глобальной памяти.

Иерархия планирования: Thread Block Clusters

В архитектуре Norper традиционная иерархия планирования потоков (Grid → Block → Thread) расширена новым промежуточным уровнем абстракции — кластерами блоков потоков (*Thread Block Clusters*). Эта парадигма позволяет группировать несколько блоков потоков на этапе компиляции и аппаратно гарантировать их совместное (кооперативное) планирование на физически соседних SM. Потоки внутри одного кластера получают возможность аппаратной барьерной синхронизации и прямого использования технологии DSMEM, что позволяет обрабатывать крупные фрагменты графов вычислений локально, не выгружая промежуточные результаты в общую память устройства.

3.1.3 Архитектура Ады Лавлейс (сентябрь 2022)

Архитектура Ada Lovelace представляет собой вычислительную платформу, аппаратно оптимизированную для ресурсоемкого графического рендеринга, симуляции физики света и нейросетевого инференса. Структурная парадигма архитектуры строится на глубокой специализации вычислительных блоков в рамках потоковых мультипроцессоров и оптимизации путей передачи данных.

Вычислительное ядро и специализированные сопроцессоры

Фундаментальным вычислительным узлом является потоковый мультипроцессор (SM), объединяющий блоки общего и узкоспециализирован-

ного назначения:

- **Ядра CUDA (Универсальные ALU):** Блоки, реализующие архитектуру параллельных вычислений и обеспечивающие одновременное (конкурентное) выполнение инструкций с плавающей запятой (FP32) и целочисленных операций (INT32).
- **Тензорные ядра (Tensor Cores):** Аппаратные матричные сопроцессоры, адаптированные для вывода нейросетей и обработки алгоритмов искусственного интеллекта. Они поддерживают расширенный спектр форматов данных, включая тензорный формат FP8, что позволяет значительно повысить пропускную способность при сохранении необходимой точности в задачах глубокого обучения.
- **Ядра трассировки лучей (RT Cores):** Выделенные аппаратные блоки для расчетов пересечения лучей с ограничивающими объемами (BVH) и треугольниками. Включают интегрированные подмодули: *Opacity Micromap Engine* для аппаратного исключения расчетов прозрачной геометрии (например, листвы) и *Displaced Micro-Mesh Engine* для генерации высокодетализированной геометрии на аппаратном уровне без перегрузки основных конвейеров и подсистемы памяти.

Аппаратная перекомпоновка и медиа-движки

Для борьбы с нерегулярными вычислительными нагрузками в архитектуру внедрены специализированные аппаратные планировщики:

- **Shader Execution Reordering (SER):** Аппаратный механизм динамического переупорядочивания потоков (threads) на лету. В задачах с высокой дивергенцией (когда лучи отражаются в разных направлениях и требуют разных шейдеров), SER группирует потоки с аналогичными инструкциями или схожим доступом к памяти. Это превращает нерегулярную нагрузку в последовательную, максимизируя утилизацию вычислительных блоков мультипроцессора.
- **Ускоритель оптического потока (Optical Flow Accelerator, OFA):** Выделенный аппаратный блок машинного зрения, который анализирует последовательные кадры и вычисляет векторы движения пикселей. Опираясь на эти данные, тензорные ядра могут генерировать полностью новые кадры методами ИИ, минуя традиционный конвейер рендеринга.
- **Двойные аппаратные кодировщики (Dual NVENC):** Интегрированные медиа-процессоры с аппаратной поддержкой кодирования

и декодирования формата AV1, способные разделять кадр на части для параллельной обработки.

Подсистема памяти и локализация данных

Архитектура памяти спроектирована с упором на максимальное удержание данных внутри чипа:

- **Масштабированный кэш второго уровня (L2 Cache):** В архитектуре реализован радикально увеличенный объем унифицированной кэш-памяти L2. Это структурное решение гарантирует, что массивы данных BVH, необходимые для трассировки лучей, и весовые коэффициенты нейросетей остаются в высокоскоростной памяти графического процессора. Это минимизирует количество транзакций по шине к подсистеме видеопамати (VRAM), снижая энергопотребление и задержки (latency) при доступе к данным.

3.1.4 Архитектура Ampere (2020)

Архитектура Ampere представляет собой унифицированную платформу, архитектурно сбалансированную для удовлетворения требований как традиционных графических рабочих нагрузок, так и интенсивных вычислений в области искусственного интеллекта (ИИ) и высокопроизводительных вычислений (HPC). Основной фокус архитектуры — повышение энергоэффективности, параллелизма и введение новых уровней точности вычислений.

Вычислительное ядро и потоковые мультипроцессоры (SM)

Потоковый мультипроцессор (SM) в архитектуре Ampere подвергся значительной переработке для обеспечения большей гибкости конвейеров:

- **Разделенные пути данных (Datapaths):** Архитектура внедряет параллельные конвейеры для выполнения инструкций. Один путь выделен исключительно для операций с плавающей запятой (FP32), в то время как второй может динамически переключаться между целочисленными операциями (INT32) и операциями FP32 в пределах одного такта. Это удваивает теоретическую пиковую производительность FP32 по сравнению с архитектурами предыдущих поколений.
- **Тензорные ядра (Tensor Cores):** Аппаратные блоки матричного умножения оснащены поддержкой новых форматов данных: *TF32* (*Tensor Float 32*) и *Bfloat16*. Формат TF32 позволяет обрабатывать данные FP32 с диапазоном FP32, но с точностью FP16, что обеспечи-

вает значительное ускорение обучения нейросетей без модификации программного кода.

- **Механизм структурной разреженности (Structural Sparsity):** Тензорные ядра получили аппаратную поддержку разреженных матриц с соотношением 2:4 (на каждые четыре значения в векторе аппаратура ожидает два нулевых). Оборудование автоматически сжимает данные и пропускает вычисления с нулями, удваивая математическую пропускную способность и снижая требования к пропускной способности памяти при инференсе нейронных сетей.
- **Ядра трассировки лучей (RT Cores):** Аппаратные блоки, выполняющие параллельные вычисления пересечений лучей с треугольниками и ограничивающими объемами (BVH). В архитектуре реализовано одновременное выполнение вычислений трассировки лучей, операций рендеринга (затенения) и тензорных вычислений.

Иерархия памяти и управление данными

Подсистема памяти Ampere спроектирована для обеспечения непрерывного потока данных к высокопроизводительным SM:

- **Асинхронное копирование (Asynchronous Copy):** На уровне SM реализованы новые инструкции для асинхронного копирования данных непосредственно из глобальной памяти в разделяемую память (Shared Memory), минуя регистровый файл. Это высвобождает регистры и снимает нагрузку с ALU во время транзакций памяти.
- **L2 Cache Residence Controls:** Архитектура предоставляет программируемые элементы управления на уровне кэша L2. Это позволяет явно назначать фрагментам данных (например, часто используемым матрицам или структурам BVH) приоритет для постоянного хранения (residency) в кэше L2, минимизируя дорогостоящие обращения к глобальной памяти.

Мультиарендность и виртуализация

Для корпоративных сред и центров обработки данных в архитектуру внедрен аппаратный механизм разделения ресурсов:

- **Multi-Instance GPU (MIG):** Аппаратная технология, позволяющая изолировать и разбивать один физический графический процессор на несколько (до семи) независимых инстансов. Каждый инстанс получает гарантированные и полностью изолированные на аппаратном уровне ресурсы: сегменты кэша L2, пропускную способность па-

мента, потоковые мультипроцессоры и блоки кодирования. Это обеспечивает строгую изоляцию отказов (fault isolation) и предсказуемую производительность при параллельном выполнении разнородных задач.

3.1.5 Архитектура Pascal (2016)

Архитектура Pascal представляет собой фундаментальный этап в эволюции графических процессоров, ознаменовавший переход к аппаратному ускорению высокопроизводительных вычислений (HPC) и алгоритмов глубокого обучения. В ее основе лежат радикальные изменения в организации памяти, коммуникационных интерфейсах и вычислительной точности.

Вычислительное ядро и потоковые мультипроцессоры (SM)

Потоковый мультипроцессор в архитектуре Pascal (в частности, в старшем ядре GP100) был структурно реорганизован для повышения эффективности планирования инструкций и максимизации утилизации регистров:

- **Оптимизированная структура SM:** Мультипроцессор разделен на два независимых вычислительных блока (вместо четырех в архитектуре Maxwell), каждый из которых имеет свой собственный буфер инструкций, планировщик (warp scheduler) и диспетчер. Это упрощает логику диспетчеризации и увеличивает долю площади кристалла, отводимой непосредственно под арифметико-логические устройства (ALU) и регистровый файл.
- **Аппаратная поддержка смешанной точности (Mixed-Precision Computing):** Архитектура впервые внедрила нативную аппаратную поддержку вычислений с половинной точностью (FP16). Инструкции FP16 могут упаковываться попарно (векторизация) и выполняться на ALU одинарной точности, что позволяет удвоить пропускную способность по сравнению с FP32. Это стало критически важным механизмом для ускорения обучения нейронных сетей.

Инновации в подсистеме памяти и адресации

Для преодоления ограничений пропускной способности (решения проблемы «стены памяти») архитектура Pascal использует передовые технологии физической компоновки и виртуализации данных:

- **Память HBM2 (High Bandwidth Memory):** Использование технологии 2.5D компоновки, при которой многослойные стеки памяти

размещаются на одной кремниевой подложке (interposer) с графическим процессором. Это решение позволило многократно увеличить пропускную способность памяти (до 720 ГБ/с в первых реализациях) и существенно снизить задержки по сравнению с дискретной памятью GDDR5.

- **Унифицированная память и аппаратный страничный обмен (Page Migration Engine):** Внедрение выделенного аппаратного контроллера, поддерживающего обработку ошибок отсутствия страницы памяти (page faults) со стороны GPU. Это обеспечило создание истинно единого виртуального адресного пространства между CPU и GPU. При обращении графического процессора к данным, находящимся в оперативной памяти хоста, контроллер автоматически, страница за страницей, мигрирует необходимые данные в локальную память VRAM по требованию (on-demand), абстрагируя программиста от ручного управления пересылками.

Механизмы коммуникации и планирования задач

Архитектура пересмотрела подходы к масштабированию многопроцессорных систем и разделению вычислительных ресурсов:

- **Высокоскоростной интерконнект NVLink:** Интеграция первого поколения проприетарной шины для когерентного и прямого взаимодействия между графическими процессорами (GPU-to-GPU), минуя корневой комплекс PCIe. NVLink обеспечил радикальное расширение полосы пропускания обмена данными, что является обязательным условием для эффективного распределенного обучения моделей (model parallelism) в кластерах.
- **Вытесняющая многозадачность (Compute/Instruction-Level Preemption):** Аппаратный механизм тонкогранулярного планирования, позволяющий прерывать выполнение вычислительных задач (compute kernels) на уровне отдельных инструкций процессора. При прерывании GPU аппаратно сохраняет полный контекст выполнения задачи в память, уступает ресурсы более приоритетному процессу, а затем восстанавливает контекст и продолжает работу. Это гарантирует предсказуемую задержку отклика системы и устраняет проблему монополизации мультипроцессоров длительными ядрами.

3.1.6 Архитектура Maxwell (2014)

Архитектура Maxwell стала знаковым этапом в развитии графических процессоров NVIDIA, сместив акцент с простого наращивания количе-

ства вычислительных блоков на радикальное повышение энергетической эффективности и оптимизацию использования ресурсов каждого транзистора. Основные инновации сосредоточены в области внутренней логики планирования и глубокой модернизации подсистемы памяти.

Реорганизация потоковых мультипроцессоров (SMM)

В архитектуре Maxwell была представлена новая структура мультипроцессора — SMM (Maxwell Streaming Multiprocessor), разработанная для достижения максимальной производительности на ватт:

- **Секционирование и логика планирования:** Каждый SMM разделен на четыре отдельных блока обработки, каждый из которых обладает собственным буфером инструкций, планировщиком (warp scheduler) и диспетчером. В отличие от предыдущих итераций, такая жесткая привязка ресурсов к конкретным группам CUDA-ядер позволила упростить логику управления и существенно снизить энергопотребление накладных расходов планирования.
- **Эффективность CUDA-ядер:** Несмотря на меньшее количество ядер на один SM (128 против 192 в Kepler), каждый SMM обеспечивает значительно более высокую полезную нагрузку за счет оптимизации путей передачи инструкций и улучшенного заполнения конвейеров, что позволяет одному ядру Maxwell выполнять до 1.35 раза больше работы при той же частоте.
- **Разделяемая память и кэш L1:** В SMM разделяемая память (Shared Memory) была физически отделена от кэша L1, что увеличило доступный объем ресурсов для высокопараллельных задач и устранило конфликты при распределении памяти, характерные для комбинированных блоков.

Оптимизация подсистемы памяти и сжатия данных

Для эффективной работы в условиях ограниченной ширины шины памяти, архитектура Maxwell внедряет передовые методы сокращения избыточности данных:

- **Третье поколение дельта-сжатия цвета (Delta Color Compression):** Графический процессор получил аппаратную поддержку более эффективных алгоритмов сжатия без потерь. Это позволяет записывать в видеопамять только разницу (дельту) между значениями соседних пикселей, что эффективно увеличивает пропускную способность памяти до 25-30% за счет снижения объема передаваемого трафика.

- **Масштабированный кэш второго уровня (L2 Cache):** В архитектуре реализовано значительное увеличение объема кэша L2 (до 2 МБ в чипах среднего сегмента, что в 8 раз больше предшественников). Это позволяет удерживать значительно больше рабочих данных на кристалле, сокращая количество энергозатратных обращений к внешней памяти VRAM.

Специализированные аппаратные блоки и алгоритмы освещения

Архитектура Maxwell привнесла новые возможности для вычислений в реальном времени и обработки медиаконтента:

- **Voxel Global Illumination (VXGI):** Аппаратная поддержка алгоритмов воксельного глобального освещения. Процессор способен эффективно переводить геометрию сцены в воксельную структуру для быстрого просчета отраженного света и мягких теней.
- **Обновленный NVENC:** Интеграция выделенного кодировщика с поддержкой формата H.265 (HEVC). Это позволило аппаратно обрабатывать видео сверхвысокого разрешения (4K) с минимальными задержками и без нагрузки на основные вычислительные CUDA-ядра.
- **Multi-Pixel Programming Sampling:** Технология, оптимизирующая работу блоков растеризации для поддержки новых режимов сглаживания и повышения качества выборки пикселей при рендеринге сложных сцен.

3.1.7 Архитектура Tesla (2006)

Архитектура Tesla представляет собой первую в индустрии полностью унифицированную вычислительную архитектуру графических процессоров. Главным концептуальным сдвигом стал отказ от отдельных аппаратных блоков для обработки вершинных и пиксельных шейдеров в пользу массива универсальных скалярных процессоров, способных выполнять любой тип вычислительной нагрузки.

Унифицированная архитектура и скалярная обработка

В основе Tesla лежит отказ от традиционной векторной обработки данных в пользу мелкозернистой скалярной архитектуры:

- **Унифицированные шейдерные ядра:** Вместо выделенных аппаратных ресурсов для разных стадий графического конвейера, Tesla использует единый массив потоковых процессоров (Streaming Processors, SP). Это позволяет динамически перераспределять вычислительную

мощность в зависимости от текущей нагрузки, устраняя простои оборудования.

- **Скалярное исполнение инструкций:** В отличие от предыдущих архитектур, оперировавших векторами (например, RGBA), процессоры Tesla работают со скалярными величинами. Это упрощает компиляцию кода и повышает эффективность использования ресурсов, так как системе не требуется заполнять все компоненты вектора для выполнения операции.

Организация потоковых мультипроцессоров (SM)

Вычислительные ресурсы Tesla сгруппированы в иерархическую структуру, оптимизированную для параллельного выполнения тысяч потоков:

- **Потоковый мультипроцессор (SM):** Каждый SM в архитектуре Tesla содержит 8 скалярных процессоров (SP), два специальных функциональных блока (SFU) для вычисления трансцендентных функций и блок управления потоками.
- **Разделяемая память (Shared Memory):** Важнейшим нововведением стало появление локальной памяти объемом 16 КБ внутри каждого SM. Эта память имеет крайне низкие задержки и позволяет потокам внутри одного блока обмениваться данными напрямую, что фактически превратило GPU в полноценный параллельный процессор.
- **Механизм выполнения Warps:** Архитектура ввела понятие «варпа» (warp) — группы из 32 потоков, которые выполняют одну и ту же инструкцию одновременно (SIMT — Single Instruction, Multiple Threads). Это позволило скрыть задержки обращения к памяти за счет мгновенного переключения между активными группами потоков на аппаратном уровне.

Подсистема памяти и модель ввода-вывода

Для поддержки общих вычислений Tesla внедрила новые механизмы взаимодействия с системными ресурсами:

- **Глобальное адресное пространство:** Tesla обеспечила возможность прямой адресации памяти, что позволило реализовать поддержку языков программирования высокого уровня, таких как C, через программно-аппаратный стек CUDA.
- **Текстурный и константный кэши:** Несмотря на универсальность, архитектура сохранила специализированные кэши для чтения дан-

ных, оптимизированные под конкретные паттерны доступа (например, двумерную локальность в текстурах), что существенно ускоряет работу графических и научных приложений.

3.2 Вычислительные возможности Nvidia CUDA

Видеокарты NVIDIA CUDA, относящиеся к разным микроархитектурам, отличаются количеством ядер различного назначения. Далее представлены основные характеристики вычислительных возможностей различных поколений Nvidia CUDA.

Микроархитектура Fermi

- В каждом мультипроцессоре (SM) содержится:
 - 32 ядра CUDA, предназначенные для операций с целыми числами и числами с плавающей точкой;
 - 16 блоков загрузки и сохранения данных (LD/ST);
 - 4 блока специального назначения (SFU), используемые для вычисления сложных арифметических функций.
- Планирование потоков осуществляется двумя планировщиками варпов (Warp Scheduler), поддерживающими одновременное выполнение независимых инструкций (dual-issue).
- Организация памяти: регистровый файл объёмом 128 КБ; кэш первого уровня (L1) — 16 КБ или 48 КБ; разделяемая память — 48 КБ или 16 КБ (конфигурация может варьироваться).
- Ключевая особенность: базовая структура SM и поддержка dual-issue для повышения производительности.

Микроархитектура Kepler

- **Мультипроцессор SMX:** содержит 192 ядра CUDA (организованные в 12 групп по 16 ядер), 64 вычислительных блока для операций с двойной точностью (FP64), 32 блока загрузки и сохранения данных (LD/ST), а также 32 блока специального назначения (SFU).
- **Планировщики:** оснащён 4 планировщиками варпов (Warp Scheduler), каждый из которых имеет по два блока выдачи команд.
- **Память:** регистровый файл объёмом 256 КБ; кэш первого уровня (L1) может быть сконфигурирован как 16, 48 или 32 КБ; разделяемая память также доступна в вариантах 48, 16 или 32 КБ; кроме того, имеется 48 КБ кэша для констант (кэш только для чтения).

Микроархитектура Maxwell

- **Мультипроцессор SMM:** состоит из четырёх блоков, каждый из которых содержит 32 ядра CUDA, 8 блоков загрузки/сохранения (LD/ST) и 8 блоков специального назначения (SFU).
- **Память:** на каждый блок приходится 64 КБ регистрового файла; отдельно выделено 90 КБ разделяемой памяти (Shared Memory).
- **Особенности:** модульная организация, двукратное повышение производительности на ватт и в 1.4 раза — на единицу площади кристалла по сравнению с предыдущими поколениями.

Микроархитектура Pascal

- **Мультипроцессор (SM):** включает два вычислительных блока с удвоенным регистровым файлом и увеличенным количеством блоков для операций с двойной точностью (FP64).
- **Память:** 64 КБ разделяемой памяти (Shared Memory), а также объединённый кэш текстур и первого уровня (Texture/L1 cache).
- **Особенности:** переход на техпроцесс 16 нм FinFET, что даёт улучшение производительности в расчёте на ватт.

Микроархитектура Turing

- **Мультипроцессор (SM):** состоит из четырёх блоков; каждый блок включает:
 - 32 ядра для вычислений с одинарной точностью (FP32);
 - 32 ядра для целочисленных операций (INT);
 - 8 ядер для двойной точности (FP64);
 - 2 тензорных ядра (Tensor Core);
 - 8 блоков загрузки/сохранения (LD/ST);
 - 1 блок специальных функций (SFU).
- **Тензорные ядра:** предназначены для обработки матриц размером 4×4 (входные данные в формате FP16/FP32, выходные — FP32/FP64), обеспечивая ускорение до 9.3 раза в задачах глубокого обучения.
- **Память:** 128 КБ общей памяти (L1/Shared Memory), причём до 96 КБ может быть выделено под разделяемую память.
- **Особенности:** независимое планирование потоков (каждый поток имеет собственный счётчик команд), техпроцесс 12 нм FinFET, наличие тензорных ядер для задач искусственного интеллекта.

3.3 Потокковая модель

Вычислительная платформа CUDA базируется на идее мультипроцессора и модели исполнения SIMT (Single Instruction Multiple Threads) — одна команда управляет множеством потоков.

При работе многопоточной программы на графическом процессоре CUDA все потоки разбиваются на блоки, а внутри блоков — на варпы (группы по 32 потока), причём все потоки одного варпа выполняют одну и ту же команду. Группа блоков выполняется на одном потоковом мультипроцессоре, а распределением нагрузки между ними занимается планировщик.

Программа, которая выполняется одновременно на нескольких блоках, называется ядром (kernel).

Отличительной чертой архитектуры CUDA является блочно-сеточная организация, которая нехарактерна для обычных многопоточных приложений. Эта организация показана на рисунке 1. При этом драйвер CUDA самостоятельно управляет распределением ресурсов устройства между потоками.

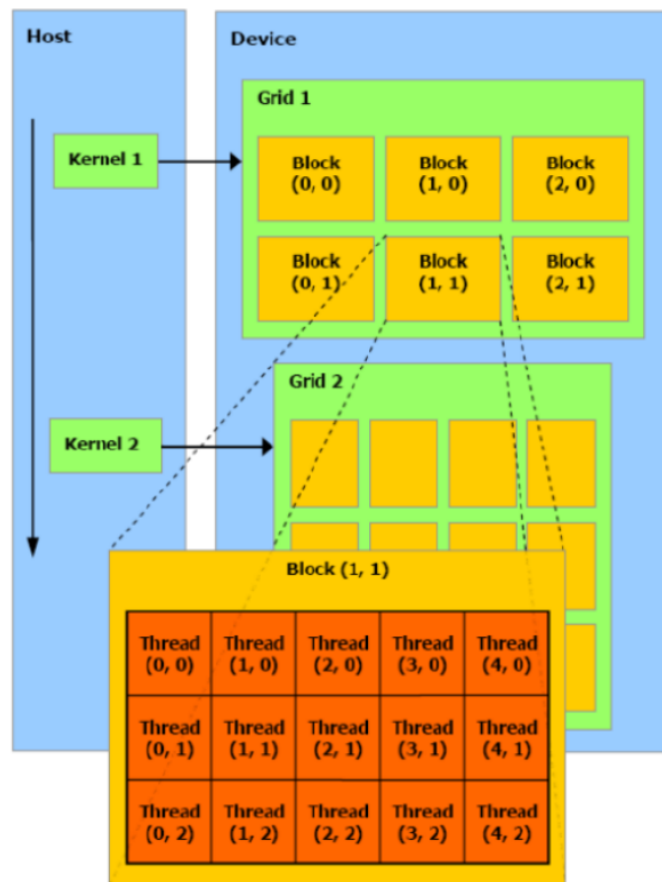


Рис. 1: Организация потоков

3.4 Устройство памяти

Графический процессор оснащён собственной глобальной памятью, которая не совпадает с оперативной памятью центрального процессора.

При выполнении кода на видеокарте обращение возможно лишь к её собственной памяти. Следовательно, для проведения вычислений и получения итогового результата требуется скопировать данные из оперативной памяти CPU в память GPU, а затем — обратно. Кроме того, каждый мультипроцессор помимо глобальной памяти располагает собственными кэшами, а также текстурной, константной, разделяемой памятью и так далее. Структура организации памяти в программах CUDA показана на рисунке 2.

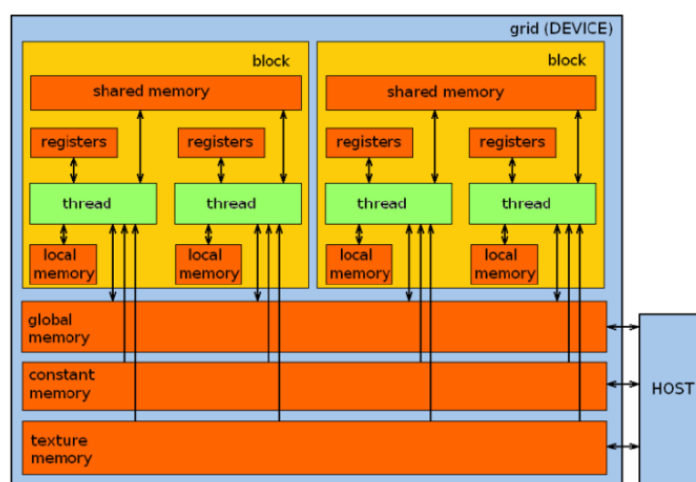


Рис. 2: Схема организации памяти CUDA-программы

3.5 Модели памяти

В архитектуре CUDA различают шесть типов памяти:

1. Регистры

Компилятор по возможности старается размещать все локальные переменные функций именно в регистровой памяти. На каждый мультипроцессор приходится 8192 32-разрядных регистров, которые распределяются между нитями, выполняющимися на этом мультипроцессоре. Доступ к регистрам является самым быстрым.

2. Локальная память

Это небольшой объем памяти, доступный только одному потоковому процессору. В неё попадают локальные данные, имеющие слишком большой размер. Физически локальная память устроена так же, как глобальная, и работает с той же скоростью.

3. Глобальная память

Глобальная память обладает большим объёмом (от 256 Мбайт до 4 Гбайт). Она доступна для произвольного чтения и записи всем мультипроцессорам, а также центральному процессору (хосту). Однако эта память очень медленная и не кэшируется, поэтому количество обращений к ней следует минимизировать. Глобальная память физически расположена на хосте, тогда как разделяемая память — непосредственно на видеокарте. Из-за этого накладные расходы на передачу данных по шине для глобальной памяти выше, чем для разделяемой, хотя сами типы памяти по своей природе идентичны.

4. Разделяемая память

Разделяемая память не кэшируется, но работает быстро. Её рекомендуется использовать в качестве управляемого кэша. На один мультипроцессор выделяется всего 16 КБ разделяемой памяти. Она служит для взаимодействия между потоками, управляется непосредственно разработчиком и обладает малой задержкой. Отличительная особенность: разделяемая память адресуется одинаково для всех задач внутри блока, то есть её можно применять для обмена данными только между потоками одного блока.

5. Константная память

Константная память хранит неизменяемые данные, доступные на уровне всей сетки (grid). Её объём составляет 64 КБ, и физически она не отделена от глобальной памяти. Однако благодаря использованию кэшей и механизма широковещательной рассылки (broadcast) этот тип памяти может повысить производительность за счёт снижения трафика между процессором и памятью. Из-за небольшого объёма в ней имеет смысл размещать лишь небольшое количество часто используемых данных.

6. Текстурная память

Текстурная память похожа на константную: через текстурный блок возможны только операции чтения. Физически она также не отделена от глобальной. Как и константная, позволяет ускорить работу благодаря системе кэшей. Её отличительная черта — оптимизация текстурного кэша для двумерной пространственной локальности.

При вычислениях на модели в виде матрицы, когда соседние элементы взаимодействуют друг с другом, часто возникает необходимость обращаться к элементам окрестности некоторого элемента матрицы. С точки зрения адресации, элементы окрестности не расположены

в памяти рядом друг с другом. Текстурная память позволяет кэшировать данные, основываясь на их пространственной, а не адресной локальности, то есть на том, что данные расположены близко в двумерном пространстве.

3.6 Модель вычислений на GPU

Программа, предназначенная для выполнения на GPU с поддержкой NVIDIA CUDA, называется ядром (kernel). Ядро запускается одновременно на сетке, состоящей из блоков, каждый из которых включает несколько нитей (потоков). Количество блоков и число нитей в блоке задаются в момент вызова ядра. Все нити выполняют один и тот же код. Чтобы обеспечить разное поведение отдельных нитей, используется информация о номере блока и нити, которая доступна каждой нити.

С точки зрения графического процессора, понятия «поток» не существует — все команды, поступающие на GPU, обрабатываются в единой очереди для всех потоков.

Планировщик способен сглаживать порядок выполнения команд, одновременно запуская копирование данных с хоста, выполнение ядра и обратное копирование на хост. Если ядро загружает менее 50% вычислительных ресурсов GPU, планировщик может запустить следующее ядро из другого потока, если оно готово к исполнению.

Для оптимизации времени выполнения задач в разных потоках важно учитывать, что команды на GPU выполняются в том порядке, в котором они были вызваны на хосте. Это означает, что не всегда эффективно полностью заполнять один поток задачами, затем второй и так далее. Более рациональный подход — равномерно распределять задачи по всем потокам.

Например, можно сначала отправить все команды в первый поток, а затем во второй. В таком случае команды второго потока будут ждать завершения команд первого. Альтернативный вариант — чередовать команды: отправить первую команду в первый поток, первую — во второй, затем вторую — в первый, вторую — во второй и так далее. Такое равномерное распределение позволяет повысить производительность, поскольку планировщик может одновременно запускать операции копирования и выполнения ядер.

3.7 Компиляция программы

Программы для видеокарт NVIDIA CUDA создаются на основе других языков программирования, в частности, используется расширение языка

C, известное как CUDA C.

Для компиляции таких программ применяется компилятор nvcc, который входит в комплект инструментов разработчика. Данный набор инструментов, а также библиотеку CUDA можно загрузить с официального сайта NVIDIA.

3.8 Планировщик задач

Поток (stream) в CUDA представляет собой логическую цепочку зависимых асинхронных операций, которые выполняются независимо от операций в других потоках. Благодаря потокам можно отправлять команды CUDA на графический процессор в том порядке, который определён внутри контекста каждого отдельного потока.

С точки зрения GPU, потоки не существуют как самостоятельные объекты — все команды, поступающие на устройство, выстраиваются в общую для всех потоков очередь. Знание этой особенности может помочь при оптимизации производительности.

Планировщик способен сглаживать порядок исполнения: он может одновременно запускать копирование данных с хоста на устройство, копирование с устройства на хост и выполнение ядра. При вызове ядра можно указать, в какой поток его поместить. По умолчанию все ядра направляются в поток 0, который является синхронным по отношению к хосту. Если ядро загружает менее 50% вычислительных ресурсов GPU, планировщик может запустить следующее ядро из другого потока при условии его готовности к выполнению.

Планировщик задач в CUDA — это механизм, управляющий исполнением потоков на GPU. Он отвечает за распределение ресурсов (вычислительных ядер, памяти и т.д.) между потоками и блоками. На рисунке 3 показан планировщик задач для архитектуры Kepler.

Kepler SM (SMX)

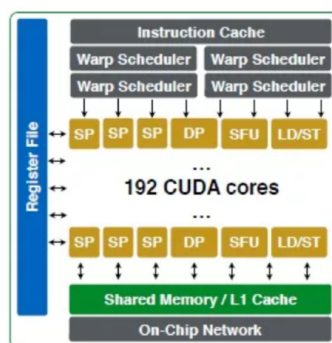


Рис. 3: Планировщик задач в архитектуре Kepler

4 Суперкомпьютерный центр «Политехнический»

4.1 Состав

В настоящий момент пользователям суперкомпьютерного центра «Политехнический» предоставлены следующие вычислительные мощности:

- 612 узлов кластера «Политехник – РСК Торнадо»;
- 56 узлов кластера «Политехник – РСК Торнадо», оснащённых ускорителями NVIDIA K-40;
- 288 узлов вычислительной системы с высокой степенью многопоточности «Политехник – РСК ПетаСтрим».

4.2 Характеристики

- Один узел кластера «Политехник – РСК Торнадо» оснащён:
 - двумя процессорами Intel Xeon E5-2697 v3 с тактовой частотой 2.60 ГГц;
 - 64 ГБ оперативной памяти.
- Один узел кластера «Политехник – РСК Торнадо» с графическими ускорителями NVIDIA K-40 включает:
 - два процессора Intel Xeon E5-2697 v3 @ 2.60 ГГц;
 - 64 ГБ RAM;
 - два ускорителя Nvidia Tesla K40x с 12 ГБ видеопамати GDDR.
- Один узел вычислителя «Политехник – РСК ПетаСтрим» содержит:
 - один процессор Intel Xeon Phi 5120D с частотой 1.10 ГГц;
 - 8 ГБ оперативной памяти.

В качестве коммуникационной сети на всех доступных узлах используется InfiniBand со скоростью 56 Гбит/с (FDR). Кроме того, на всех узлах доступна параллельная файловая система Lustre объёмом около 1 петабайта.

По умолчанию пользователю предоставляется доступ к узлам `tornado`. Доступ к узлам других типов выдаётся по отдельному запросу.

4.3 Технология подключения

Для получения доступа зарегистрированному пользователю к суперкомпьютерному центру (СКЦ) следует применить любой терминальный клиент, поддерживающий протокол SSH. Это обеспечивает подключение к удалённому терминалу для работы с ресурсами СКЦ.

В рамках работы использовалась следующая процедура подключения:

1. От администрации СКЦ были получены публичный и приватные ключи в виде файлов, а также логин пользователя для доступа к системе.
2. Подготовка к подключению:
 - Для организации подключения к СКЦ использовалась программа `OpenSSH`. На устройства с операционной системой `MacOS Tahoe` она предустановлена.
 - Для обмена файлами использовался встроенный в операционную систему инструмент `SFTP`.
3. Для подключения к СКЦ необходимо ввести команду:

```
1 ssh -i .ssh/private-key login@login1.hpc.spbstu.ru
```

где:

- `login` — логин для подключения к СКЦ;
 - `login1.hpc.spbstu.ru` — адрес узла СКЦ;
 - `-i .ssh/private-key` — по до приватного файла-ключа.
4. Для передачи файлов между используемой машиной и СКЦ требуются следующие параметры:

Для входа:

- Имя хоста: `login1.hpc.spbstu.ru`
- Порт: `22`
- Логин: `login`.

Для обмена данными:

- Навигация по удаленной системе:
 - `pwd` — показать текущую удалённую директорию.
 - `cd имя_папки` — перейти в папку на сервере.
 - `ls` — список файлов на сервере (обычный или с ключами `ls -la`).

- Навигация по локальной системе:
 - `lpwd` — показать текущую локальную директорию.
 - `lcd путь` — сменить локальную директорию (куда файлы будут скачиваться/откуда загружаться).
 - `lls` — список файлов на вашем компьютере.
- Передача файлов:
 - `get файл` — скачать файл с сервера в текущую локальную папку.
 - `get -r папка` — скачать папку целиком (рекурсивно).
 - `put локальный_файл` — загрузить файл с вашего компьютера на сервер.
 - `put -r папка` — загрузить папку со всем содержимым.

В результате настройки было осуществлено подключение к СКЦ по протоколу SSH. На рисунке 4 представлена стартовая страница при подключении к СКЦ.

```

Last login: Mon Apr 13 17:04:35 2026 from 148.222.186.164

/##### /##### /##### /##### /##### /$ /##### /$ /$
/$__ $ /$__ $ /$__ $ /$__ $ /$__ $ | $__ $ | $__ $ | $__ $
| $$_\_\_/ | $$_\_\_/ | $$_\_\_/ | $$_\_\_/ | $$_\_\_/ | $$_\_\_/ | $$_\_\_/ | $$_\_\_/
| ##### | $ | $ | $ | $ | $ | $ | $ | $ | $ | $ | $ | $ | $ | $
\_____ $ | $ | $ | $ | $ | $ | $ | $ | $ | $ | $ | $ | $ | $
/$$_\_\_/ | $$_\_\_/ | $$_\_\_/ | $$_\_\_/ | $$_\_\_/ | $$_\_\_/ | $$_\_\_/ | $$_\_\_/
| #####/ | #####/ | #####/ | #####/ | #####/ | #####/ | #####/ | #####/
\_____ / \_____ / \_____ / \_____ / \_____ / \_____ / \_____ /

Slurm partitions:
* tornado : nodes: 612
            cpu: 2 x Intel Xeon CPU E5-2697 v3 @ 2.60GHz
            cores/hwthreads: 28 / 56
            mem: 64G
            net: 56Gbps FDR Infiniband
* tornado-k40 : nodes: 56
            cpu: 2 x Intel Xeon CPU E5-2697 v3 @ 2.60GHz
            cores/hwthreads: 28 / 56
            co-processor: 2 x Nvidia Tesla K40x
            co-processor mem: 12G
            mem: 64G
            net: 56Gbps FDR Infiniband

Storage: 1 PB Lustre FS

List of available software: module avail

tm4u42@login1:~
$ █
  
```

Рис. 4: Стартовая страница при подключении к СКЦ

5 Постановка решаемой задачи

Дано:

- массив P — полигон $(n \times n)$ целых чисел.
- A, B, C — координаты вершин треугольника в локальной системе координат.
- R — радиус окружности.

Требуется: вычислить K_1 — число треугольников в массиве P , K_2 — число окружностей в массиве P .

Ограничения:

- границы контура треугольника в массиве P — 2;
- границы контура окружности в массиве P — 3;
- распределение треугольников и окружностей в массиве P выполнено по случайному закону;
- $R = 100$, треугольник ABC равносторонний, длина стороны > 100 ;
- треугольники и окружности в массиве P могут пересекаться;
- значения элементов массива P отличны от 2 и 3.

6 Алгоритм решения задачи

Для вычисления K_1 и K_2 в условиях возможного пересечения контуров наиболее устойчивым методом является **Обобщенное преобразование Хафа**.

Предобработка данных

Создадим две бинарные маски из исходного массива P :

- $M_T(i, j) = 1$, если $P[i, j] = 2$ (контур треугольника);
- $M_C(i, j) = 1$, если $P[i, j] = 3$ (контур окружности).

Формирование эталонов (ядер)

Определим множества относительных координат точек идеальных контуров:

1. Δ_C : координаты (dx, dy) точек окружности $x^2 + y^2 = 100^2$.
2. Δ_T : координаты (dx, dy) сторон равностороннего треугольника относительно его центра масс.

Процедура голосования

Для каждого типа фигуры выполняется следующий цикл накопления:

```
1:  $Acc[n][n] \leftarrow 0$  ▷ Матрица аккумулятора
2: for  $x$  от 1 до  $n$  do
3:   for  $y$  от 1 до  $n$  do
4:     if  $M[x, y] == 1$  then
5:       for каждого  $(dx, dy) \in \Delta$  do
6:          $cx = x - dx, cy = y - dy$ 
7:         if  $(cx, cy)$  в границах массива then
8:            $Acc[cx, cy] \leftarrow Acc[cx, cy] + 1$ 
9:         end if
10:      end for
11:    end if
12:  end for
13: end for
14:  $K \leftarrow$  количество локальных максимумов в  $Acc > \text{Порог}$ 
```

Обоснование параметров

Так как фигуры могут пересекаться, часть пикселей одного контура может быть заменена пикселями другого. Установка порога обнаружения

на уровне 80 – 90% от общего числа точек эталона позволяет компенсировать эти потери и верно вычислить K_1 и K_2 .

6.1 Глобальная память

1. Распределение потоков

- Общее число элементов матрицы аккумулятора равно $N = n \times n$. Запускается одномерная 1D-сетка потоков.
- Индекс потока вычисляется как:

$$threadId = blockIdx.x \cdot blockDim.x + threadIdx.x$$

- Каждый поток обрабатывает элементы с индексами $idx = threadId + i \cdot T$, где T — общее число потоков в сетке, при условии $idx < N$.
- Для каждого найденного контурного пикселя поток выполняет атомарную запись результатов в глобальную память.

2. Алгоритм голосования

- Координаты текущего пикселя вычисляются через линейный индекс:

$$row = \frac{idx}{n}, \quad col = idx \pmod{n}$$

- Если $P[row][col]$ соответствует контуру, выполняется цикл по шаблону Δ :

$$Acc[row - dx][col - dy] = \sum atomicAdd(\dots)$$

6.2 Разделяемая память

1. Использование Shared Memory

- Блок потоков копирует массив смещений шаблона Δ из глобальной памяти в разделяемую память блока для ускорения доступа:

$$shared_delta[k] = delta[k]$$

- Синхронизация потоков внутри блока осуществляется через барьер `__syncthreads()`.

2. Сбор данных

- Каждый поток вычисляется один элемент результирующей матрицы аккумулятора:

$$Acc[row][col] = \sum_{k=0}^{N_{\Delta}-1} M[row + shared_delta[k].x][col + shared_delta[k].y]$$

где M — бинарная маска соответствующей фигуры.

- Данный подход исключает использование атомарных операций, так как каждый поток пишет строго в свою ячейку памяти.

7 Описание эксперимента

Цель эксперимента — оценка зависимости времени решения задачи от объема обрабатываемых данных и параметров запуска, связанных с GPU.

Для исследования времени решения задачи в зависимости от параметров были выбраны следующие параметры:

- Используемая память: глобальная и разделяемая.
- Количество блоков: 1, 10, 100, 1000, 10000.
- Количество потоков в блоке: 32, 64, 128, 256, 512, 1024.
- Размеры полигона P : 1000×1000 , 5000×5000 , 10000×10000 .
- Количество элементов в матрице: 10^4 , 10^6 , $2.5 \cdot 10^7$.
- Параметры фигур: радиус окружности $R = 100$, сторона треугольника > 100 .

Для каждой комбинации параметров было проведено 100 измерений с помощью событий CUDA и взято среднее значение. Временные затраты на копирование данных между Host и Device в итоговых графиках учитывались отдельно от времени выполнения вычислительных ядер.

8 Анализ результатов

На рисунке 5 представлены результаты эксперимента.

```
tm4u42@login1:~/cudaLab
$ cat results.csv
N,BLOCKS,THREADS,MODE,TIME_MS
[1000,1,32,GLOBAL,0.0125722
1000,1,32,SHARED,0.0127299
1000,1,64,GLOBAL,0.0122877
[1000,1,64,SHARED,0.0127603
1000,1,128,GLOBAL,0.0126976
1000,1,128,SHARED,0.0129126
1000,1,256,GLOBAL,0.0124013
[1000,1,256,SHARED,0.0132022
1000,1,512,GLOBAL,0.0385862
1000,1,512,SHARED,0.0412342
1000,1,1024,GLOBAL,0.0395523
1000,1,1024,SHARED,0.0410522
1000,10,32,GLOBAL,0.0394851
1000,10,32,SHARED,0.0406611
1000,10,64,GLOBAL,0.0378397
1000,10,64,SHARED,0.0423786
1000,10,128,GLOBAL,0.0384022
1000,10,128,SHARED,0.0423152
1000,10,256,GLOBAL,0.0386854
1000,10,256,SHARED,0.0425104
1000,10,512,GLOBAL,0.0386832
1000,10,512,SHARED,0.042936
1000,10,1024,GLOBAL,0.0410598
1000,10,1024,SHARED,0.0434595
1000,100,32,GLOBAL,0.040592
1000,100,32,SHARED,0.043871
1000,100,64,GLOBAL,0.0407558
1000,100,64,SHARED,0.0434669
1000,100,128,GLOBAL,0.0419709
1000,100,128,SHARED,0.0438243
1000,100,256,GLOBAL,0.0440563
1000,100,256,SHARED,0.050935
1000,100,512,GLOBAL,0.0714512
1000,100,512,SHARED,0.0897785
1000,100,1024,GLOBAL,0.128687
1000,100,1024,SHARED,0.167456
1000,1000,32,GLOBAL,0.0498125
1000,1000,32,SHARED,0.054488
1000,1000,64,GLOBAL,0.0696451
1000,1000,64,SHARED,0.0764647
1000,1000,128,GLOBAL,0.0905373
1000,1000,128,SHARED,0.110057
1000,1000,256,GLOBAL,0.139268
1000,1000,256,SHARED,0.268222
1000,1000,512,GLOBAL,0.25661
1000,1000,512,SHARED,0.653751
1000,1000,1024,GLOBAL,1.00327
1000,1000,1024,SHARED,1.37284
5000,1,32,GLOBAL,0.0141885
5000,1,32,SHARED,0.0138099
5000,1,64,GLOBAL,0.0127594
5000,1,64,SHARED,0.0131206
5000,1,128,GLOBAL,0.0106467
5000,1,128,SHARED,0.0121907
5000,1,256,GLOBAL,0.0132765
5000,1,256,SHARED,0.0133827
5000,1,512,GLOBAL,0.0406163
5000,1,512,SHARED,0.0434467
5000,10,64,SHARED,0.0416291
5000,10,128,GLOBAL,0.0380317
5000,10,128,SHARED,0.042327
5000,10,256,GLOBAL,0.0404326
5000,10,256,SHARED,0.0425936
5000,10,512,GLOBAL,0.038631
5000,10,512,SHARED,0.0422986
5000,10,1024,GLOBAL,0.0400598
5000,10,1024,SHARED,0.0418765
5000,100,32,GLOBAL,0.0392531
5000,100,32,SHARED,0.0438182
5000,100,64,GLOBAL,0.038848
5000,100,64,SHARED,0.0418067
5000,100,128,GLOBAL,0.0412784
5000,100,128,SHARED,0.0424272
5000,100,256,GLOBAL,0.0442541
5000,100,256,SHARED,0.0505011
5000,100,512,GLOBAL,0.0729878
5000,100,512,SHARED,0.0905382
5000,100,1024,GLOBAL,0.127613
5000,100,1024,SHARED,0.162727
5000,1000,32,GLOBAL,0.0503216
5000,1000,32,SHARED,0.0532016
5000,1000,64,GLOBAL,0.070088
5000,1000,64,SHARED,0.0771894
5000,1000,128,GLOBAL,0.0891504
5000,1000,128,SHARED,0.107325
5000,1000,256,GLOBAL,0.137785
5000,1000,256,SHARED,0.2568
5000,1000,512,GLOBAL,0.253697
5000,1000,512,SHARED,0.629355
5000,1000,1024,GLOBAL,1.02667
5000,1000,1024,SHARED,1.38824
10000,1,32,GLOBAL,0.0129165
10000,1,32,SHARED,0.0131142
10000,1,64,GLOBAL,0.0128381
10000,1,64,SHARED,0.0125901
10000,1,128,GLOBAL,0.0119498
10000,1,128,SHARED,0.0128195
10000,1,256,GLOBAL,0.0121283
10000,1,256,SHARED,0.0134909
10000,1,512,GLOBAL,0.0397283
10000,1,512,SHARED,0.0407216
tm4u42@login1:~/cudaLab
$ █
```

Рис. 5: Результаты эксперимента

В таблицах 1-3 представлены усредненные результаты работы алго-

ритма с использованием глобальной памяти.

Таблица 1: Усредненные результаты измерения времени исполнения программы для матрицы 1000×1000 и глобальной памяти (мс)

Число потоков в блоке	Число блоков			
	1	10	100	1000
32	0.0134	0.0407	0.0410	0.0501
64	0.0112	0.0410	0.0395	0.0702
128	0.0115	0.0388	0.0399	0.0894
256	0.0128	0.0389	0.0429	0.1399
512	0.0391	0.0400	0.0727	0.2538
1024	0.0390	0.0391	0.1298	1.0008

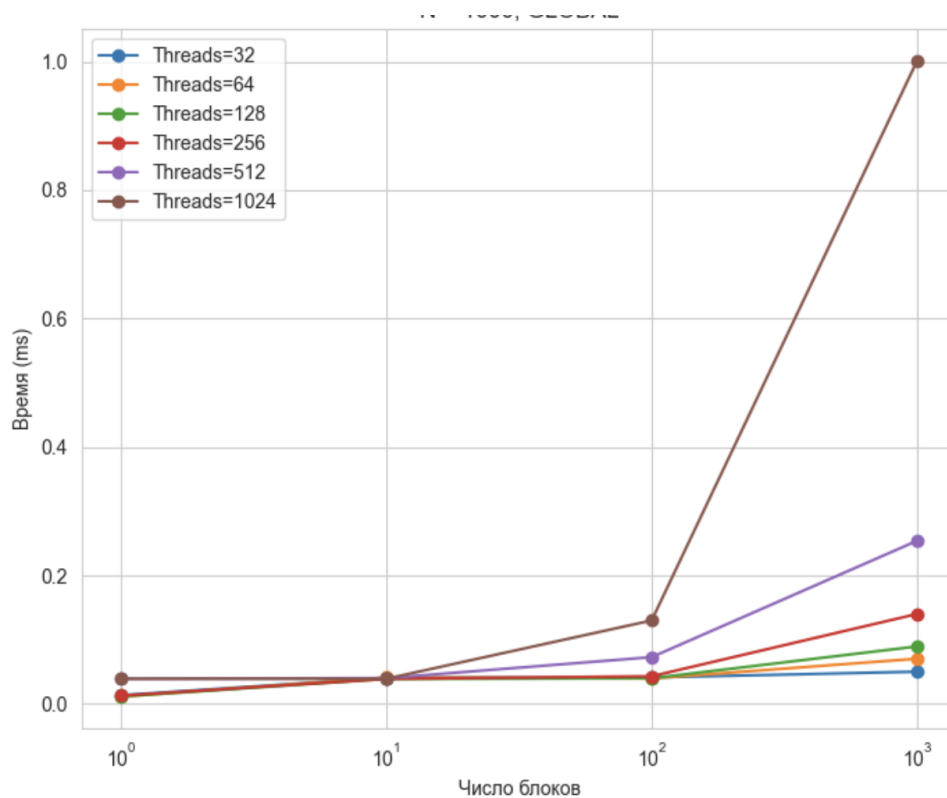


Рис. 6: График результатов времени исполнения программы для матрицы 1000×1000

Таблица 2: Усредненные результаты измерения времени исполнения программы для матрицы 5000×5000 и глобальной памяти (мс)

Число потоков в блоке	Число блоков			
	1	10	100	1000
32	0.0131	0.0399	0.0398	0.0485
64	0.0130	0.0391	0.0400	0.0695
128	0.0129	0.0388	0.0418	0.0886
256	0.0112	0.0398	0.0446	0.1397
512	0.0395	0.0396	0.0728	0.2525
1024	0.0381	0.0402	0.1279	1.0233

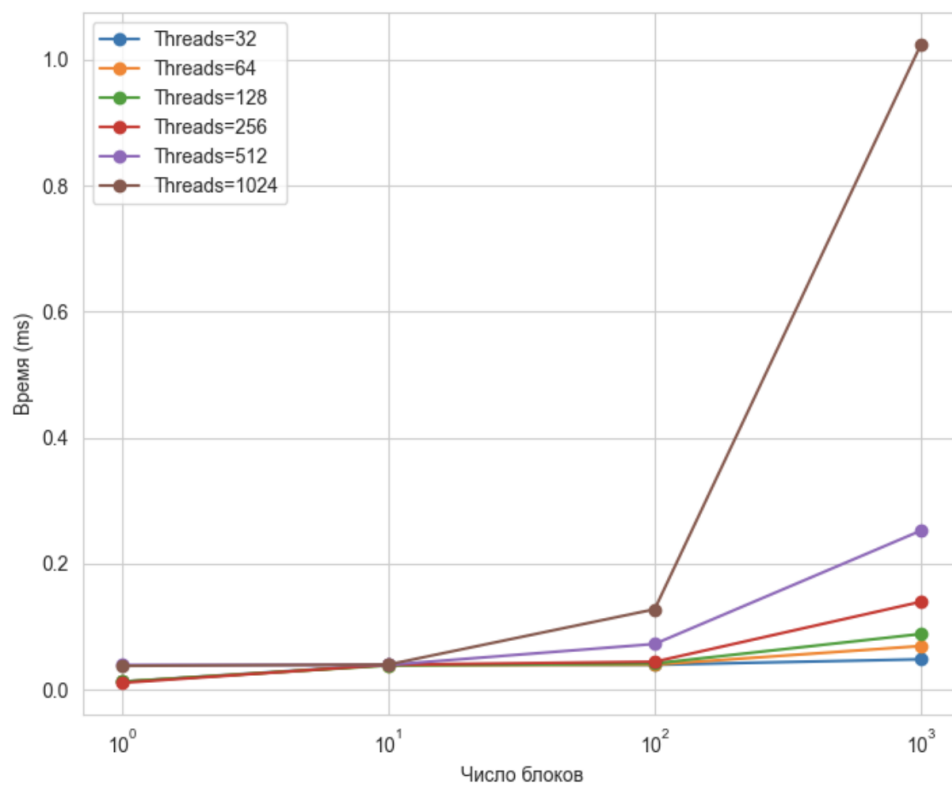


Рис. 7: График результатов времени исполнения программы для матрицы 5000×5000

Таблица 3: Усредненные результаты измерения времени исполнения программы для матрицы 10000×10000 и глобальной памяти (мс)

Число потоков в блоке	Число блоков			
	1	10	100	1000
32	0.0132	0.0389	0.0415	0.0488
64	0.0109	0.0392	0.0393	0.0695
128	0.0110	0.0388	0.0408	0.0894
256	0.0130	0.0398	0.0442	0.1379
512	0.0396	0.0406	0.0726	0.2520
1024	0.0387	0.0399	0.1276	1.0155

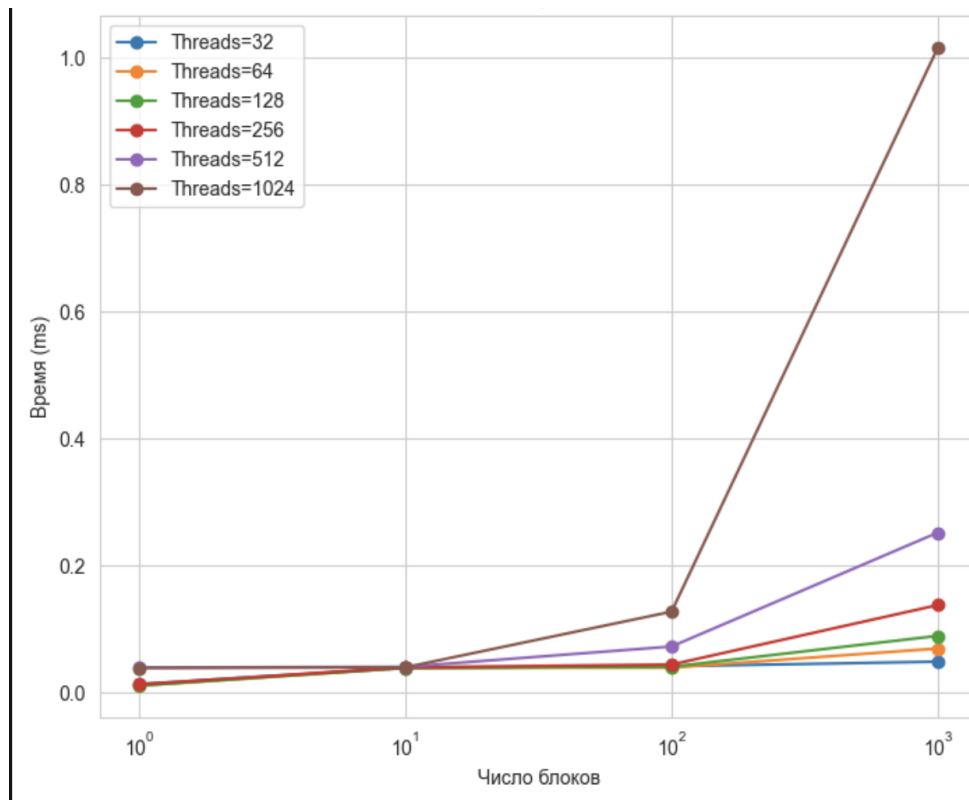


Рис. 8: График результатов времени исполнения программы для матрицы 10000×10000

В таблицах 4-6 представлены усредненные результаты работы алгоритма с использованием глобальной и разделяемой памяти.

Таблица 4: Усредненные результаты измерения времени исполнения программы для матрицы 1000×1000 и разделяемой памяти (мс)

Число потоков в блоке	Число блоков			
	1	10	100	1000
32	0.0135	0.0430	0.0422	0.0532
64	0.0131	0.0425	0.0407	0.0786
128	0.0115	0.0409	0.0447	0.1087
256	0.0130	0.0437	0.0524	0.2678
512	0.0420	0.0444	0.0912	0.6505
1024	0.0414	0.0432	0.1691	1.3692

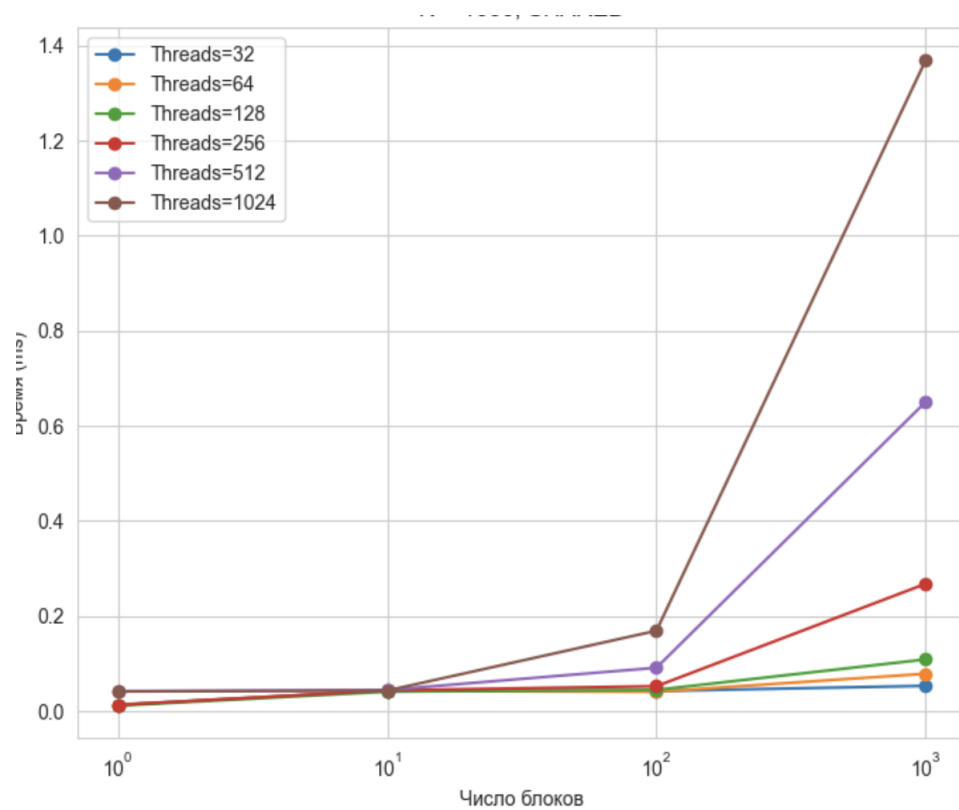


Рис. 9: График результатов времени исполнения программы для матрицы 1000×1000

Таблица 5: Усредненные результаты измерения времени исполнения программы для матрицы 5000×5000 и разделяемой памяти (мс)

Число потоков в блоке	Число блоков			
	1	10	100	1000
32	0.0120	0.0437	0.0415	0.0514
64	0.0133	0.0408	0.0423	0.0758
128	0.0133	0.0424	0.0447	0.1062
256	0.0111	0.0420	0.0528	0.2579
512	0.0429	0.0416	0.0904	0.6266
1024	0.0431	0.0430	0.1627	1.3838

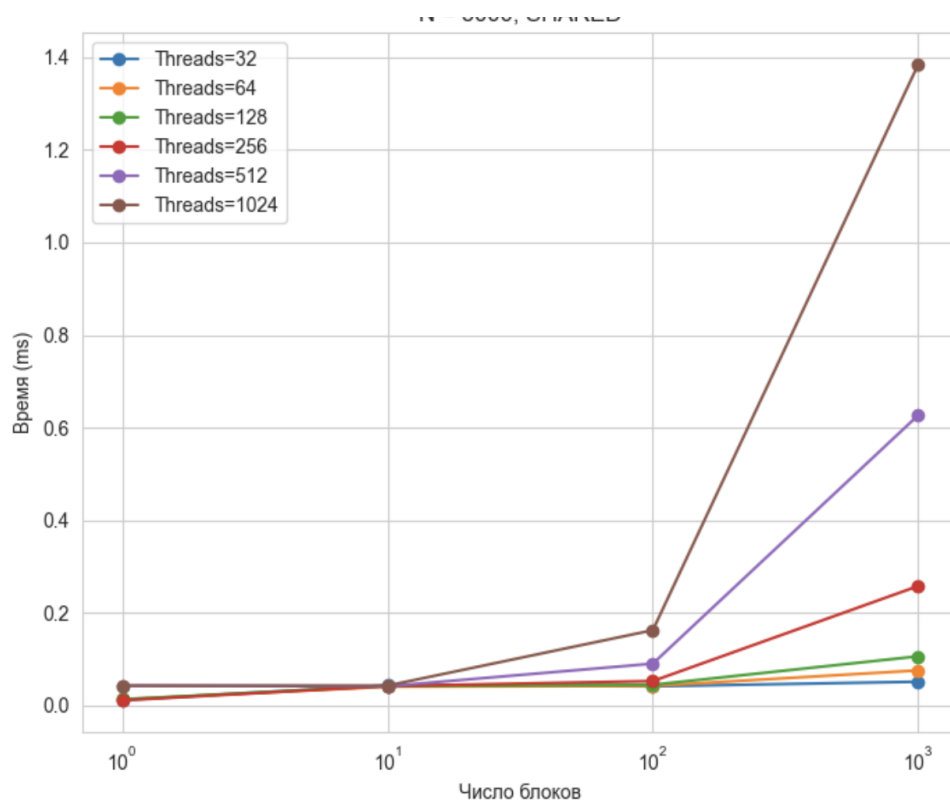


Рис. 10: График результатов времени выполнения программы для матрицы 5000×5000

Таблица 6: Усредненные результаты измерения времени выполнения программы для матрицы 5000×5000 и разделяемой памяти (мс)

Число потоков в блоке	Число блоков			
	1	10	100	1000
32	0.0124	0.0410	0.0427	0.0546
64	0.0132	0.0413	0.0419	0.0758
128	0.0118	0.0418	0.0436	0.1061
256	0.0120	0.0407	0.0534	0.2546
512	0.0434	0.0430	0.0900	0.6158
1024	0.0432	0.0425	0.1624	1.3465

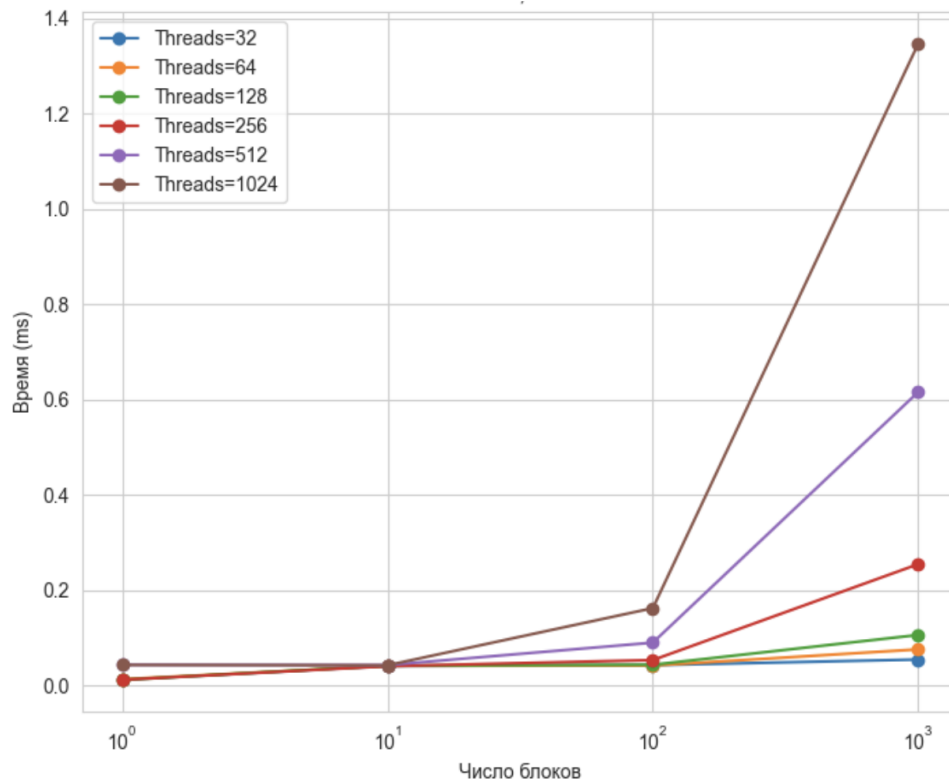


Рис. 11: График результатов времени исполнения программы для матрицы 5000×5000

Зависимость от размера полигона

Для анализа влияния размера полигона на время выполнения сопоставим результаты, полученные для $N = 1000, 5000, 10\,000$ (таблицы 1–6). В отличие от интуитивных ожиданий, в исследованном диапазоне время исполнения практически не зависит от линейного размера полигона.

- **Минимальное время (глобальная память).** Для $N = 1000$ наименьшее время составило 0.0112 мс (64 потока, 1 блок); для $N = 5000$ — также 0.0112 мс (256 потоков, 1 блок); для $N = 10\,000$ — 0.0109 мс (64 потока, 1 блок). Разброс составляет менее 3% и лежит в пределах погрешности измерений.
- **Минимальное время (разделяемая память).** Аналогичная картина наблюдается и для режима совместного использования глобальной и разделяемой памяти: 0.0115 мс ($N = 1000$), 0.0111 мс ($N = 5000$), 0.0118 мс ($N = 10\,000$).
- **Максимальное время (наихудшая конфигурация).** Даже при максимальной загрузке GPU (1000 блоков, 1024 потока) время остаётся практически постоянным: глобальная память — 1.0008 мс, 1.0233 мс, 1.0155 мс; разделяемая память — 1.3692 мс, 1.3838 мс, 1.3465 мс. Ни

в одном случае не наблюдается пропорционального роста времени с увеличением N .

Зависимость от числа блоков и потоков

Результаты, представленные в таблицах 1–6, показывают, что время выполнения в исследованной задаче определяется главным образом конфигурацией запуска ядра, а не объёмом вычислений. В отличие от интенсивных вычислительных задач, где увеличение числа потоков приводит к пропорциональному сокращению времени, в нашем случае накладные расходы на планирование и синхронизацию доминируют.

- **Влияние числа потоков в блоке.** При малом количестве блоков (1–10) время практически не зависит от числа потоков, оставаясь в диапазоне 0.011 ... 0.043 мс для любой конфигурации. С ростом числа блоков чувствительность к числу потоков резко возрастает. Например, для глобальной памяти при $N = 1000$ и 1000 блоках время увеличивается с 0.0501 мс (32 потока) до 1.0008 мс (1024 потока), т.е. почти в 20 раз. Это объясняется тем, что большое число потоков в сочетании с большим числом блоков создаёт чрезмерную нагрузку на планировщик GPU и ограниченные аппаратные ресурсы (регистры, разделяемая память), что приводит к сериализации исполнения.
- **Влияние числа блоков.** При фиксированном числе потоков увеличение блоков до 100 почти не меняет время; заметный рост начинается только при 1000 блоках. Особенно сильно этот эффект выражен для конфигураций с 512 и 1024 потоками. Так, при 1024 потоках время выполнения для глобальной памяти и $N = 1000$ составляет 0.0390 мс (1 блок), 0.1298 мс (100 блоков) и 1.0008 мс (1000 блоков). Следовательно, количество блоков начинает играть негативную роль, когда суммарное число потоков превышает возможности эффективного планирования на конкретном GPU.
- **Оптимальная конфигурация.** Минимальное время во всех экспериментах достигается при использовании единственного блока и числа потоков 64 или 128. Эта комбинация избегает заметных накладных расходов на запуск блоков и синхронизацию, обеспечивая задержку на уровне 0.011 мс. Дальнейшее увеличение числа блоков или потоков не приводит к сокращению времени, а лишь добавляет паразитные затраты.

Сравнение глобальной и разделяемой памяти

Результаты сравнения двух подходов к организации памяти для всех исследованных размеров полигона сведены в таблице 7.

Таблица 7: Максимальное и минимальное время для каждого размера и типа памяти

Размер N	Тип памяти	Мин/макс	Время (мс)	Блоки	Потоки
1000	Глобальная	Максимальное	1.0008	1000	1024
		Минимальное	0.0112	1	64
	Разделяемая	Максимальное	1.3692	1000	1024
		Минимальное	0.0115	1	128
5000	Глобальная	Максимальное	1.0233	1000	1024
		Минимальное	0.0112	1	256
	Разделяемая	Максимальное	1.3838	1000	1024
		Минимальное	0.0111	1	256
10000	Глобальная	Максимальное	1.0155	1000	1024
		Минимальное	0.0109	1	64
	Разделяемая	Максимальное	1.3465	1000	1024
		Минимальное	0.0118	1	128

Анализ таблицы 7 и детальных таблиц 1–6 показывает, что в исследованной задаче использование разделяемой памяти не даёт выигрыша в производительности ни при одном из рассмотренных размеров полигона. Напротив, во всех случаях минимальное время для разделяемой памяти либо практически совпадает с глобальным, либо немного хуже, а максимальное время всегда больше для разделяемой памяти.

- **Малые накладные расходы при 1 блоке.** Для единственного блока разница во времени между глобальной и разделяемой памятью пренебрежимо мала. Это объясняется тем, что при таком запуске синхронизация и загрузка данных в разделяемую память выполняются лишь одним блоком, а сама работа настолько мала, что задержки памяти не являются узким местом.
- **Рост накладных расходов с увеличением параллелизма.** С ростом числа блоков и потоков накладные расходы разделяемой памяти резко возрастают. Например, при 1000 блоках и 1024 потоках для $N = 1000$ время для разделяемой памяти (1.3692 мс) на $\sim 37\%$ выше, чем для глобальной (1.0008 мс), а для $N = 5000$ это превышение достигает 35%. Причина в том, что дополнительные операции синхронизации и копирования данных из глобальной памяти в разделяемую становятся доминирующим фактором при большом количестве одновременно работающих блоков.
- **Отсутствие выгоды от ускоренного доступа.** Хотя время доступа к разделяемой памяти на порядок меньше, чем к глобальной,

в данной задаче каждый поток использует данные лишь однократно (или малое число раз), поэтому преимущества разделяемой памяти не успевают проявиться. Положительный эффект от быстрого доступа полностью нивелируется временем, затраченным на её инициализацию и барьерную синхронизацию.

- **Зависимость от размера полигона не выявлена.** Даже для самого большого размера $N = 10000$ разделяемая память не даёт преимущества, что отличает данную задачу от многих классических GPU-алгоритмов (например, умножения матриц или свёрток), где с ростом объёма данных выигрыш от разделяемой памяти становится заметнее.

Таким образом, в рамках проведённого эксперимента оптимальной стратегией является использование глобальной памяти с минимальным числом блоков и умеренным числом потоков. Разделяемая память лишь добавляет накладные расходы, не обеспечивая выигрыша в скорости, и её применение в аналогичных лёгких ядрах следует считать нецелесообразным.

Заключение

В ходе данной курсовой работы была изучена технология параллельного программирования на основе архитектуры NVIDIA CUDA. В теоретической части был проведён обзор архитектуры CUDA, включая её вычислительные возможности, иерархию памяти и модель исполнения на графических процессорах NVIDIA.

В практической части работы:

- Реализовано и распараллелено ядро на языке CUDA C для вычисления числа треугольников и окружностей на полигоне.
- Проведена отладка на локальной машине с поддержкой CUDA.
- Финальная серия экспериментов выполнена на суперкомпьютере «Политехнический» с использованием вычислительного узла «Политехник-РСК Торнадо» (графический ускоритель NVIDIA K40).
- Варьировались следующие параметры: размер полигона ($N = 1000, 5000, 10\,000$) число блоков (1, 10, 100, 1000), число потоков в блоке (32, 64, 128, 256, 512, 1024), тип используемой памяти (глобальная, глобальная + разделяемая).

Основные выводы по результатам эксперимента:

1. **Время выполнения не зависит от размера полигона** в исследованном диапазоне N . И глобальная, и разделяемая память показали практически одинаковые значения минимального и максимального времени для всех трёх размеров. Это свидетельствует о том, что вычислительная нагрузка на поток пренебрежимо мала, а доминируют накладные расходы на запуск и синхронизацию.
2. **Использование разделяемой памяти не дало выигрыша в производительности.** Для всех конфигураций время с разделяемой памятью было либо практически равно времени с глобальной памятью (при малом числе блоков), либо заметно выше (при 1000 блоках и большом числе потоков). Причина — в лёгком ядре время копирования данных в разделяемую память и барьерная синхронизация не окупаются более быстрым доступом.
3. **Оптимальной стратегией является минимизация параллелизма.** Минимальное время во всех случаях достигается при использовании одного блока и 64–256 потоков. Увеличение числа блоков до 1000 или потоков до 1024 приводит к многократному росту времени из-за конкуренции за ресурсы GPU и издержек планирования.

4. Конфигурация с одним блоком и умеренным числом потоков является наилучшей для исследованного класса задач. Широко распространённый подход, при котором произведение числа блоков на число потоков должно примерно равняться числу элементов, в данном случае неэффективен именно из-за крайне низкой полезной нагрузки на один поток.

Таким образом, для лёгких CUDA-ядер с малым объёмом вычислений на поток ключевым фактором становится не пропускная способность памяти или пиковая производительность GPU, а минимизация накладных расходов. Результаты работы показывают, что в подобных ситуациях следует отдавать предпочтение глобальной памяти и небольшому числу потоковых блоков, избегая избыточного распараллеливания.

Список литературы

- [1] Сандерс, Д. Технология CUDA в примерах: введение в программирование графических процессоров. — М: ДМК Пресс, 2013. — 232 с.
- [2] Краткое руководство пользователя вычислителей «Политехник - РСК Торнадо» и «Политехник - РСК Пегастрим».
- [3] Суперкомпьютерный центр — Ресурсы СКЦ. // Суперкомпьютерный центр «Политехнический». [Электронный ресурс]. — Режим доступа: URL: <https://scc.spbstu.ru/resources> (дата обращения: 20.03.2026).